



Lecture::Big Data and ElasticSearch

Luca Morandini

Head of AI Engineering – Melbourne eResearch Group

University of Melbourne

luca.morandini@unimelb.edu.au

Outline of this Lecture

Part 1: “Big data” challenges and architectures

- DBMS for distributed environments
- What distributed DBMS look like: PostgreSQL vs CouchDB vs ElasticSearch
- Consistency, Availability and Partition-tolerance in distributed environments
- Sharding

Part 2: Introduction to ElasticSearch

- Architecture
- Managing indexes
- Managing documents
- Queries

Part 1: “Big data” Challenges and Architectures

“Big data” Is Not Just About “Bigness”

The four “Vs” :

- **Volume:** yes, volume (Giga, Tera, Peta, ...) is a criteria, but not the only one
- **Velocity:** the frequency at which new data is being brought into the system
- **Variety:** the variability and complexity of data schema. The more complex and heterogenous data schema(s) you have, the higher the probability of them changing along the way
- **Veracity:** the level of trust in data accuracy (provenance); the more diverse sources you have, the more unstructured they are, the less veracity you have -especially if data reflect different levels of currency in the data sources

Big Data Calls for Ad-hoc Solutions

- While Relational DBMSs (MySQL, PostgreSQL, SQL Server, Oracle, etc) are extremely good at ensuring consistency, they rely on normalized data models that, in a world of big data (think about Veracity and Variety) can no longer be taken for granted.
- Therefore, it makes sense to use DBMSs that are built upon data models that are not relational (relational models are composed of *tables*, *columns* and *relationships* amongst tables - see *relational algebra* for a proper theory)
- While there is nothing preventing SQL to be used in distributed environments, alternative query languages have been used for distributed DBMSs, hence they are sometimes called *NoSQL DBMSs* (as *No SQL used* or *Not-Only SQL*)
- Often NoSQL DBMSs support SQL, but not the complete set of features (for instance, Elasticsearch SQL does not support joins)
- *NewSQL* DBMSs (CockroachDB, VoltDB) promise to combine the dependability of relational databases with the scalability of NoSQL databases

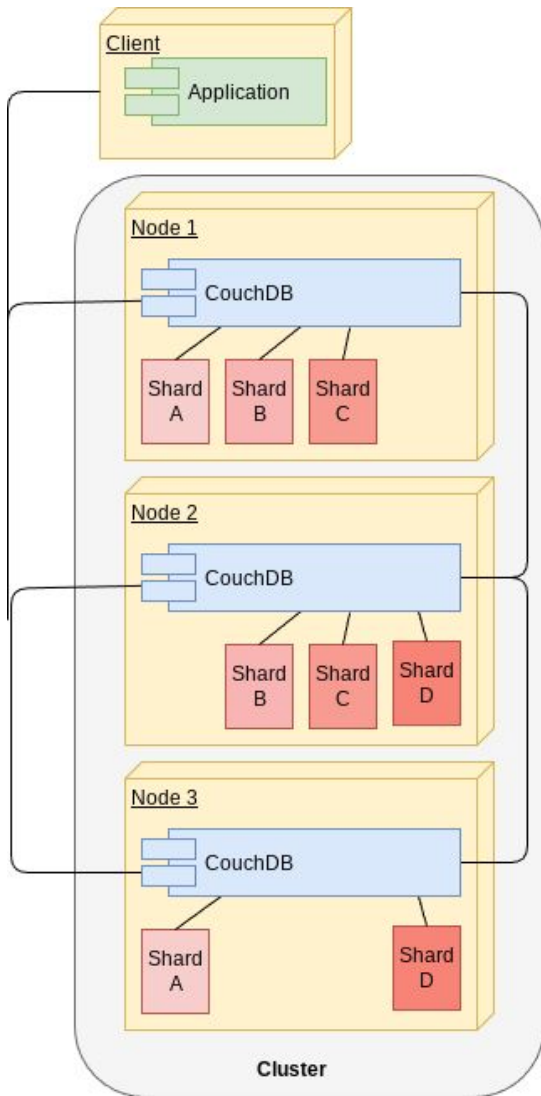
Data Models for Distributed Environments

- A **key-value store** is a DBMS that allows the retrieval of a chunk of data given a key: fast, but crude (e.g. Redis, RocksDB, Berkeley DB)
- A **BigTable DBMS** stores data in columns grouped into *column families*, with rows potentially containing different columns of the same family (e.g. Apache Cassandra, Apache Accumulo): they are more an evolution of key-values stores than of relational DBMSs
- A **Document-oriented DBMS** stores data as structured documents, usually expressed as XML or JSON (e.g. Apache CouchDB, MongoDB, ElasticSearch)

Database Clusters

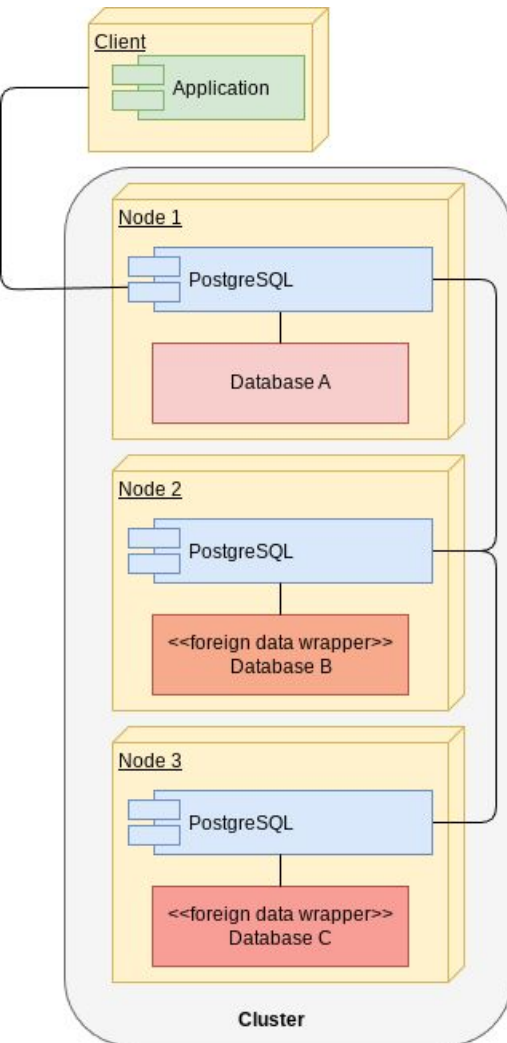
- Distributed DBMS run as *clusters* that span multiple compute nodes (VMs or physical computers)
- In general, a *cluster* is a set of instances of a software process that collectively behave like a single software process
- When, in a k8s deployment, more than one replica is declared, it does not mean that our application magically become clustered. It really depends on the application type, which is tricky with software that has side-effects, such as DBMS
- Clusters are needed to:
 - Distribute computing load over multiple nodes (to improve *performance*)
 - Store multiple copies of the same data (to achieve *redundancy*)
- Let's consider three different DBMSs (*CouchDB*, *PostgreSQL* and *ElasticSearch*) and their associated architectures

CouchDB Cluster Architecture



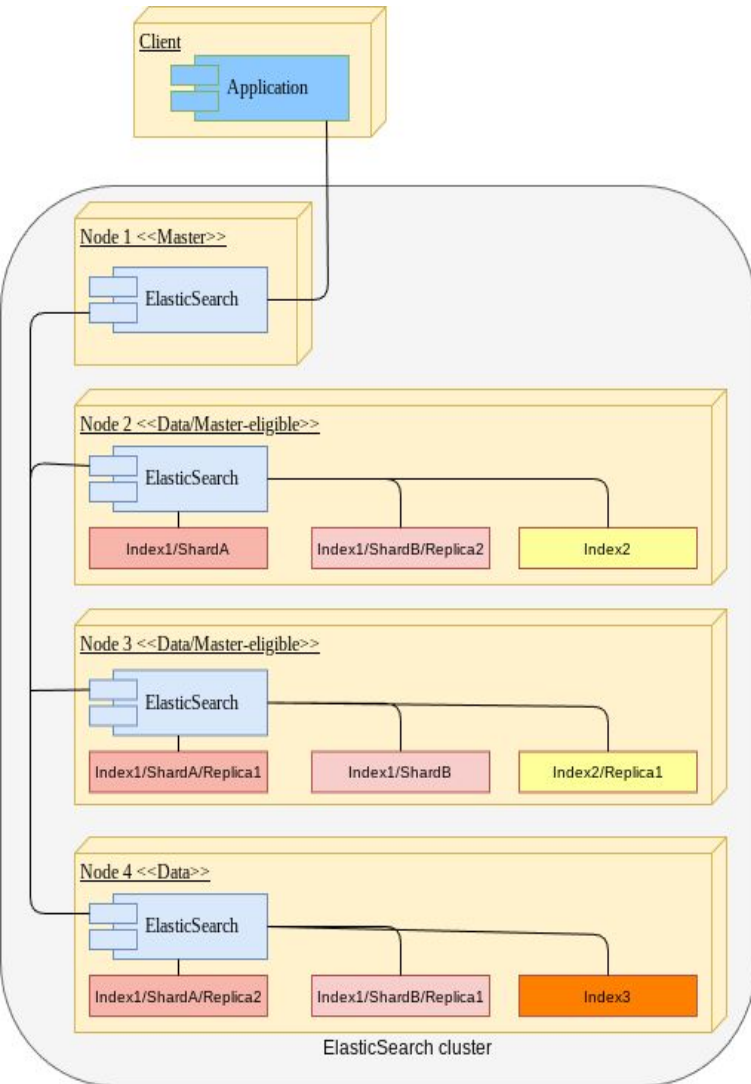
- All nodes answer requests (read or write) at the same time
- Shards (data partitions) are hosted on every node
- In this example there is:
 - 1 database
 - 3 nodes
 - 4 shards
 - 2 copies of each shard
- When a node does not contain a document (say, a document of Shard A is requested from Node 2), the node requests it from another node (say, Node 1) and returns it to the client
- In case of a failing node, the two other remaining nodes keep on working, since there is another copy of a shard to use
- Nodes can be added/removed, and their shards re-balanced automatically based on the addition/deletion of nodes (for existing databases the re-sharding has to be done manually by moving shard files and/or splitting/merging shards)

PostgreSQL Federated Database Architecture



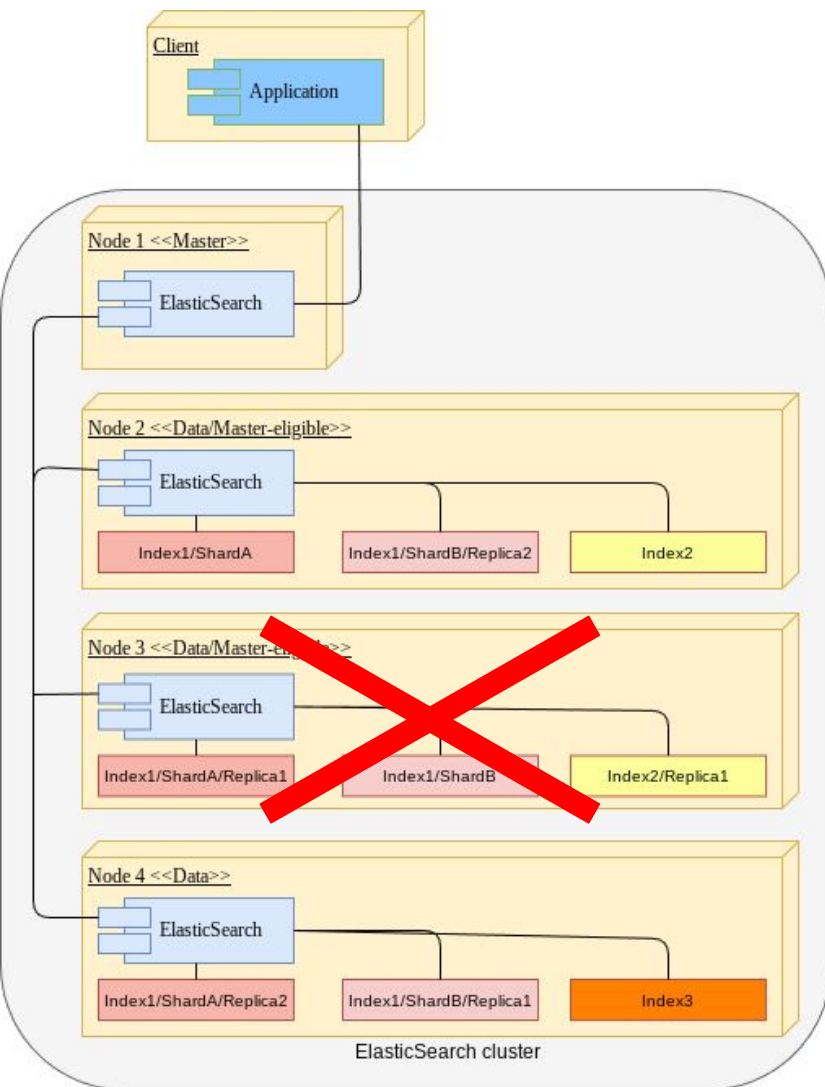
- This is just one way out of many of organizing distributed databases in PostgreSQL (see Citus, Postgres-XL, PL/Proxy)
- This is an example of a *federated database* (a database that is the composition of separated databases that look like a single one)
- There is one entry point (Node 1) for client applications, data residing on the other nodes are accessed only through Node 1
- Different databases have different tables: for instance, a *customers* table can reside on Database B while an *invoices* table can be located on Database C, and so on
- When a query requires a table that is located on Database B or C, the *foreign data wrapper* mechanism allows PostgreSQL to access those table as though they were local to Node 1
- Write requests to tables present in Database B or C are forwarded to the relevant nodes and the transaction has to succeed on all nodes
- In case of a failing node, the whole cluster may stop working (depending on which node and what data are requested)
- NOTE: in the PostgreSQL documentation, a *cluster* is a set of databases residing on the same node, not a cluster as normally intended (i.e. a set of DBMS software processes that appear as one DBMS)

ElasticSearch Cluster Architecture



- ElasticSearch has two main *node roles*: *Master* and *Data* (there are other roles of specialised nodes, but they are not relevant for this discussion)
- A node can have more than one node role
- This example shows an ES cluster composed of 4 nodes:
 - 1 master node
 - 2 master-eligible nodes that double as data nodes
 - 1 data node
- More than one node can have *master* role but only one node in a cluster can be a *master* at any given moment (the other nodes with the *master* role are *master-eligible* and can become *master* only when the current master goes down)
- Every node can act as a *coordinating node* (coordinating query execution), but only master nodes can create or delete an index and decide to move shards around
- *Indexes* (databases) can be sharded and replicated
- The indexes on this cluster are:
 - an index with 2 *primary shards* and 2 *replica shards* (Index1);
 - an index that is unsharded and has 1 replica (Index2)
 - an index that is unsharded and has no replicas (Index3)
- The same shard (be it a primary or a replica shard) cannot be duplicated on the same node
- In case of a master node failing, a *master-eligible* node takes its place
- Only the primary shard (or the whole index in case of unsharded) can be updated, with the changes propagated to the replica shards
- ElasticSearch propagates changes to replicas waiting for all replicas to acknowledge the change before considering the transaction completed
- Replica shards can be queried (read-only requests) in parallel to speed up computation

ElasticSearch Cluster “Yellow” State

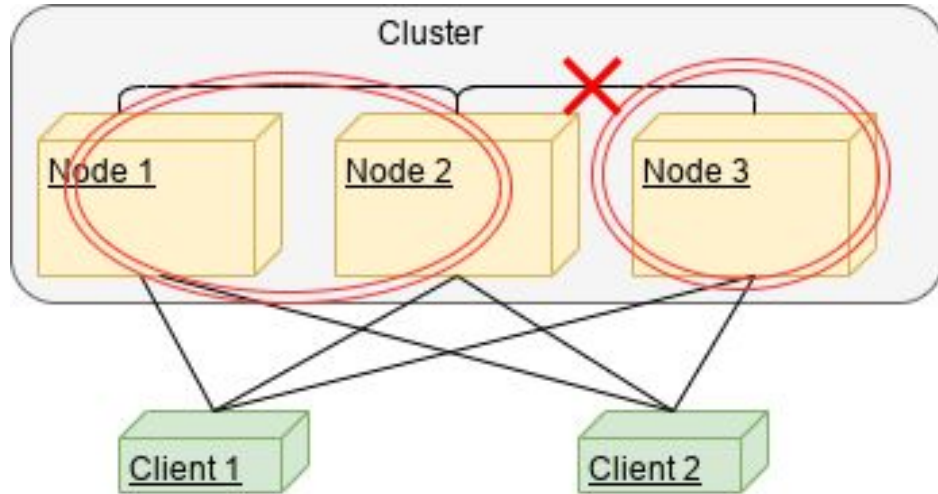


- If all primary shards are assigned but at least one replica shard is unassigned, the cluster is shown in **Yellow** status
- For instance if Node 3 were to crash:
 - Index 1 would be shown as **Yellow**
 - Index 2 would be shown as **Yellow** until the **Index2/Replica1** is recreated on Node 4, then shown as **Green**
 - Index 3 would be shown as **Green**
- **Note:** Index 1 cannot be shown in **Green** until a new node is added to the cluster (since it has a replica number of 2, Index 1 needs three nodes)
- **Note:** it is the task of the master node to coordinate the re-creation of **Index2/Replica1**

CouchDB vs PostgreSQL vs Elasticsearch

- Elasticsearch restricts write requests directed to an index to only one node at a time (the node hosting the primary shard) while every node in CouchDB can accept write requests
- Elasticsearch has elections in place to establish another master node in case one fails, and there is the possibility of not reaching a minimum number of master-eligible nodes (a quorum), hence forcing the cluster to stop accepting write requests. By contrast, there are no elections in CouchDB, even when one or more nodes fail
- In PostgreSQL foreign data wrapper a failed node hinders the functioning of the whole cluster and can lead to some data not being accessible at all
- These differences are rooted in different approaches to an unsolvable problem, a problem defined by *Brewer's CAP Theorem*

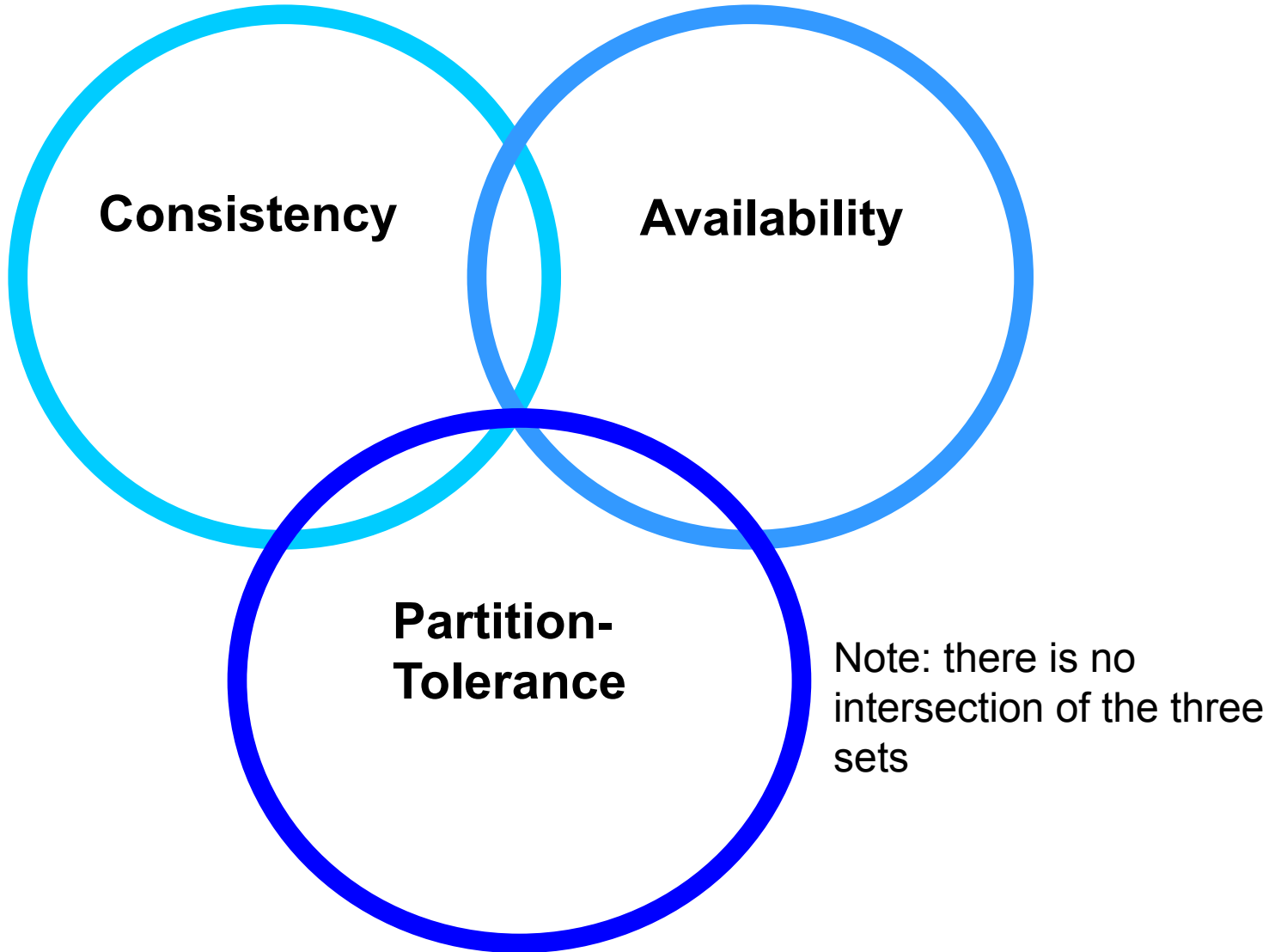
Consistency, Availability, Partition-Tolerance



- **Consistency:** every client receiving an answer receives the same (non error) answer from all nodes of the cluster
- **Availability:** every client receives a non-error answer (not necessarily the same) from every node of the cluster
- **Partition-tolerance:** the cluster keeps on operating when one or more nodes cannot communicate with the rest of the cluster

Brewer's CAP Theorem

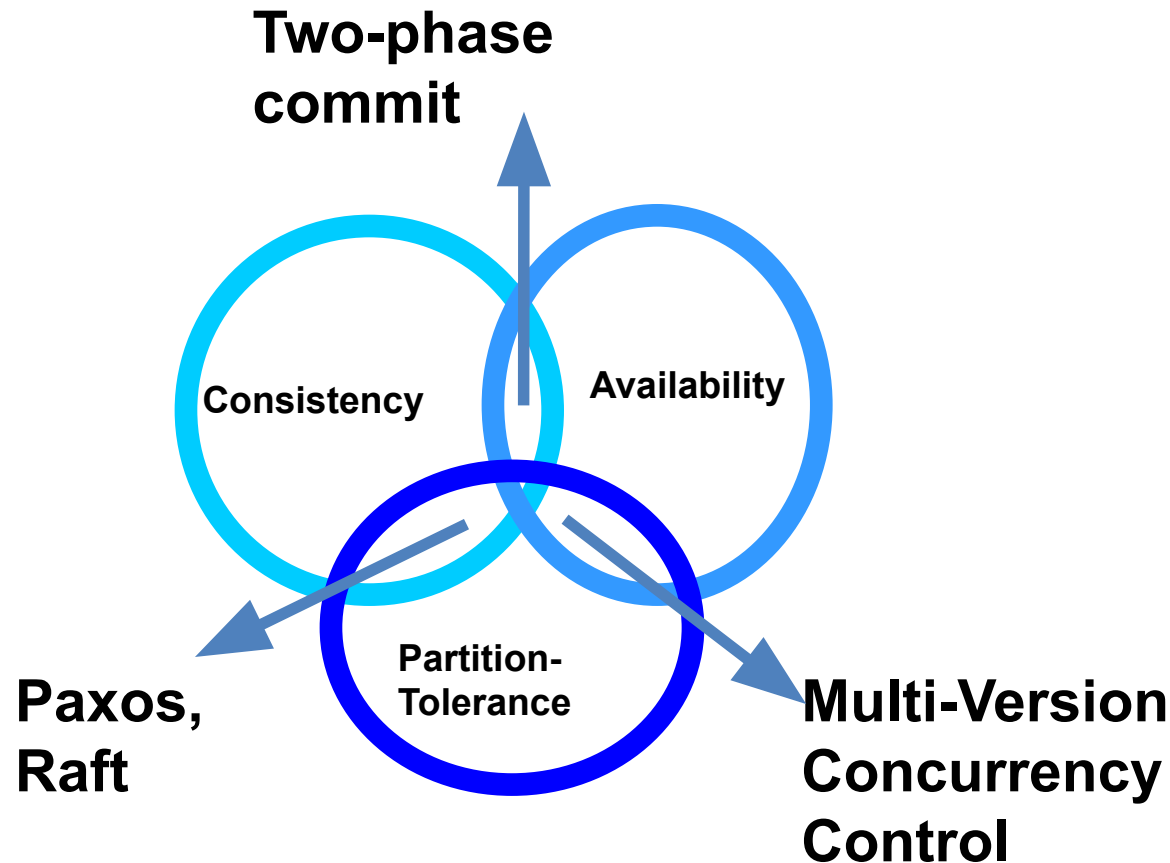
Consistency, Availability and Partition-Tolerance: pick any two...



More on Brewer's CAP Theorem

- While the theorem shows all three qualities are symmetrical, Consistency and Availability are at odds only when a Partition happens
- “Hard” network partitions (a node crashing) may be rare, but “soft” ones are not (e.g. a slow node may be considered crashed even if it is not). Ultimately, every partition is detected by a *timeout*; for instance Elasticsearch interprets a delay of 500ms from a master node as a sign that the node has crashed and an election must be held
- Network partitions can have consequences that impact the cluster as a whole, e.g. a distributed join is only complete when all subqueries return
- Traditional DBMS architectures were not much concerned with network partitions, since all data were supposed to be in a small, co-located cluster of high-quality servers
- The emphasis on numerous *commodity servers*, shared across multiple tenancies in a commercial cloud results in an increased number of hardware failures or long latency
- The CAP theorem forces us to consider trade-offs among different options

CAP Theorem and the Classification of Distributed Processing Algorithms



Consistency and Availability: Two phase commit

This is the usual algorithm used in relational DBMSs (and Elasticsearch, to some extent), it enforces consistency by:

- locking data that are within the transaction scope
- performing transactions on write-ahead logs
- completing transactions (commit) only when all nodes in the cluster have performed the transaction
- aborting transactions (rollback) when a partition is detected

This procedure entails the following:

- reduced performance (data lock)
- no availability in case of a partition
- enforced consistency (every database is kept in a consistent state)

Consistency and Partition-Tolerance: Paxos

- This family of algorithms is driven by consensus, and is both partition-tolerant and consistent
- In Paxos, every node is either a *proposer* or an *accepter* :
 - a *proposer* proposes a value (with a timestamp)
 - an *accepter* can accept or refuse it (e.g. if the *accepter* receives a more recent value)
- When a proposer has received a sufficient number of acceptances (a *quorum* is reached), and a confirmation message is sent to the *accepters* with the agreed value
- Paxos clusters can recover from partitions and maintain consistency, but the smaller part of a partition (the part that cannot reach the quorum) will not send responses to clients, hence availability is reduced
- *Raft* is a similar, but simpler, algorithm solving the same problem

Availability and Partition-tolerance: Multi-Version Concurrency Control (MVCC)

- MVCC is a method to ensure availability (every node in a cluster always accepts requests), and some sort of recovery from a partition by reconciling the single databases with *revisions* (data are not replaced, they are just given a new revision number)
- In MVCC, concurrent updates are possible without distributed locks (in *optimistic locking* only the local copy of the object is locked), since the updates have different revision numbers; the transaction that completes last will get a higher revision number, hence will be considered as the *current value*.
- In case of cluster partition and concurrent requests with the same revision number going to two partitioned nodes, both are accepted, but once the partition is solved, there would be a *conflict*... a conflict that would have to be solved somehow (CouchDB returns a list of all current conflicts, which must then be solved by the application). Think of it as something similar to a software revision control system such as *Git*.

CouchDB vs PostgreSQL vs Elasticsearch - Revisited

- **CouchDB** uses MVCC, hence it is a highly-available DBMS and maintains its availability even when a network partition happens, possibly sacrificing its consistency
- **ElasticSearch** uses a mix of two-phase commit (for replicating data from primary to replica shards) and Paxos-like behaviour (to elect a master node), hence it is consistent but not available when a network partition occurs.

To ensure in-order updates of documents, a *sequence number* and a *primary term* (a number that identifies the primary shard) has to be passed to the transaction request

- **PostgreSQL** federated databases with foreign data wrapper use a two-phase commit, hence they do not tolerate a network partition

Why Document-oriented DBMS for Big data?

While Relational DBMSs are extremely good for ensuring consistency and availability, the normalization that lies at the heart of a relational database model implies fine-grained data, which are less conducive to partition-tolerance than coarse-grained data (intuitively, related pieces of data are stored together).

Example:

- A typical contact database in a relational data model may include: a person table, a telephone table, an email table and an address table, all linked to each other.
- The same database in a document-oriented database would entail one document type only, with telephones numbers, email addresses, etc., nested as arrays in the same document.

Sharding

- Sharding is the partitioning of a database “*horizontally*”, i.e. the database rows (or documents) are partitioned into subsets that are stored on different servers. Every subset of rows is called a *shard*.
- The main advantage of a sharded database lies in the improvement of performance through the distribution of computing load across nodes. In addition, it makes it easier to move data files around, e.g. when adding new nodes to the cluster
- A sharded database allows the total database size to exceed the amount of persistent storage on any single node
- There are different sharding strategies, most notably:
 - *Hash sharding*: to distribute rows evenly across the cluster
 - *Range sharding*: similar rows (say, social media posts coming from the same area) are stored on the same shard

Replication and Sharding

- *Replication* is the action of storing the same row (or document) on different nodes to make the database fault-tolerant
- *Sharding* is the partition of data into different “chunks”
- Replication and sharding can be combined to maximize availability while maintaining a minimum level of data safety

Part 2: Introduction to Elasticsearch

Why Are We Using Elasticsearch in This Course?

- Elasticsearch (<https://www.elastic.co/elasticsearch>) is open-source, hence you can peruse the source code and see how things work
- It can be used to query data by space (geography) and time
- It is widely used
- It is a distributed DBMS and its clusters are easy to setup on Kubernetes
- It has sharding and replication
- It can store vectors (such as machine learning *features*) and execute k-nearest neighbour searches
- The HTTP API makes it easy to interact with it
- It has a powerful GUI (Kibana)

The Evolution of Elasticsearch

- It started as an HTTP- and JSON-based API around the Apache Lucene full-text search engine in 2010
- It has grown “organically”, and it is messy at times (there are often different ways to get the same result)
- It is now a document-based, distributed, scalable DBMS with powerful query languages and a ReSTful API

What is Elasticsearch Good at?

- Full-text search
- Retrieval of time-based data, such as computer logs
- Storing unstructured data
- Storing data that are changed infrequently, if ever

What is Elasticsearch Bad at?

- Storing data that are linked to one another (say, invoices and customers). In this case a relational DBMS is a better choice
- Elasticsearch does not really support joins between documents (except for parent-child relationship in Query DSL and enrich or lookup joins in ES|QL)
- Supporting multiple-record transactions (e.g. changing a set of documents in an all-or-nothing way): Elasticsearch only supports safe transactions of one record (document) at a time
- Storing data that are analysed across multiple dimensions, such as in a *data warehouse*: a dedicated OLAP engine would be a better choice
- Storing data that are changed frequently. Documents are not deleted neither updated (when updated, the old version is marked as deleted and a new version of the doc is added), which makes the size of index files grow fast

ElasticSearch Concepts

- **Index:** an index is comparable to a *database* in a relational DBMS
- **Documents:** the records (comparable to *rows* in a relational DBMS) of an ElasticSearch index. A document is expressed in JSON
- **Data stream:** a set of indexes that follow the same naming pattern and appear as one entity. This is usually (but not exclusively) done to rotate logs and other time-based information (a naming pattern could be: `nginx-*`, where the asterisk contains one of the dates the logs were collected on)
- **Shard:** horizontal partition of an index
- **Replicas:** number of copies of a shard (two replicas means there are three copies of the same document in an index: one *primary shard* and two *replica shards*)
- **Node:** a node is an instance of ElasticSearch
- **Cluster:** multiple nodes that cooperate to manage the same indexes
- **Mappings:** a schema that maps the fields of the JSON documents in an index to data types and indexing methods

ElasticSearch Components

- **ElasticSearch**: the component that does the storing and searching of documents
- **FileBeat**: the component that listens for updates in files (typically log files) and loads the updates into indexes
- **MetricBeat**: a component that monitors the state of a system and stores the metrics into indexes
- **LogStash**: a component that transforms data collected by data sources (such as FileBeat and MetricBeat) and stores the transformed data into ElasticSearch indexes
- **Kibana**: a web-based user interface that allows the query and management of an ElasticSearch cluster

More Elasticsearch Components

- ***FunctionBeat***: a component that monitors the activity of serverless environment such as AWS Lambda and stores the results into indexes
- ***HeartBeat***: a component that checks the status of services and stores the results into indexes
- ***Observability***: component that observes system metrics, analyses them, and generates insights
- ***Security***: same as the Observability component but for security
- ...and many more

ElasticSearch Clusters

- Node *roles*: every node in a cluster is assigned one or more *roles*
- There are many roles (*master*, *data*, *transform*, *ingest*, *ml*, etc), but the only necessary ones are: *master* and *data*
- A *master* node coordinates the cluster action (e.g. deciding where shards are allocated, tracking the health of nodes, creating indexes, etc)
- There could be more than one master node in a cluster but there can only be one active master node (the others are *master-eligible*)
- A *data* node that is not master-eligible manages data but cannot be elected to master or participate in elections
- When a master fails, the other master nodes hold an election to choose the new master. Replica shards hosted on the remaining nodes become primary shards to replace those on the failed nodes (if any)

ElasticSearch Scaling

- Scaling can be either *vertical* (provisioning a more powerful node and transfer the data onto it) or *horizontal* (adding more nodes to a cluster)
- When new data nodes are added to an ElasticSearch cluster some shards migrate to the new nodes, to rebalance the load
- When data nodes are removed from an ElasticSearch cluster, the primary shards of the removed nodes need to be replaced by replica shards on the remaining nodes (one of the replica shards becomes primary)

ElasticSearch Index Status

- **Green**: when there are enough nodes to host all shards (primary and replica)
- **Yellow**: when all primary shards are assigned but at least one replica shard is unassigned (the same shard cannot be stored twice on the same node). A yellow cluster keeps on working, but the required data redundancy level cannot be assured
- **Red**: when the number of working nodes falls below the required number of primary shards
- *Index state vs Cluster state*: even a single **yellow** or **red** index state makes the entire cluster state to go **yellow** or **red**

Therefore, a cluster in a “bad” state is no cause for panic, it could just be a single index that is under-replicated

ElasticSearch Shards

- ElasticSearch by default uses *hash sharding* on the document ID to ensure an equal number of documents in each shard
- Every index can have a different number of shards and replicas
- A specific *routing* value can be set during the creation of a document to put the document in a specific shard, overriding the default mechanism to implement *range sharding*
- Searches use one thread per shard (although multiple searches can happen at the same time on the same shard)
- The bigger the shard, the slower the search, but every shard consumes resources, hence the optimal number of shards depends on several criteria
- A rule of thumb for the optimal number of shards can be having about 200M documents per shard, or between 10GB - 50GB per shard
- In general it is better to have too many shards than too few, since the number of primary shards limits the maximum number of nodes that can be used to update an index; besides, databases have the tendency to grow more than anticipated

ElasticSearch Index Life-cycle

- Some data stored in ElasticSearch are time-sensitive and can grow extensively, therefore it makes sense to remove older data when they become either too old or too large
- In ElasticSearch an index can have its *life-cycle policy* defined through the following phases: **Hot**, **Warm**, **Cold**, **Frozen**, **Delete**
- Depending on their life-cycle indexes can be assigned to nodes depending on their performance (a node can be assigned a role such as: `data_hot`, `data_cold`, etc.)
- At a simpler level, one can set an index (or, rather, a *data stream*) to be *rolled over* once it reaches a certain size or is older than a given number of days, and eventually deleted
- For instance, a data stream `nginx-*` could have an index-life policy so that it is rolled-over and passed into a warm state after 10 days and deleted after 30 days (after 10 days there will be two indexes, say: `nginx-2024-01-01` and `nginx-2024-01-11`, after 20 days three indexes, after 30 days there will still be three indexes as the older one will be removed)

Creating an Index

- Index: an index must have a name, or, if a data stream, a name pattern
- Mapping: an index can have an *explicit mapping* that prescribes how the different fields have to be indexed (without an explicit mapping Elasticsearch uses *dynamic mapping*, that is, it will decide how to index fields based on some heuristics inferred from the documents as they are being inserted)
- Leaving it to Elasticsearch to use dynamic mapping may lead to inefficiencies (such as indexing too many fields), hence it is advised to provide an explicit mapping
- During its creation an index is assigned:
 - The number of shards it has to be split into
 - The number of replicas of each shard
 - The index life-cycle policy
 - The field mapping (explicit or dynamic, see above)
 - ...and many other parameters

Data Types

- The main data types in Elasticsearch include:
 - *Binary* (encoded as Base64 strings)
 - *Boolean* (true or false as JSON values)
 - *Keyword* (a string that contain a limited number of values, such as postal codes, city names, etc)
 - *Text* (a string field that is full-text indexed)
 - *Numeric* (integers, float, double, etc)
 - *Date* (timestamps expressed as either strings or numbers)
 - *Object* (not indexed, but its fields can be)
 - *Array* (there are no arrays in Elasticsearch, but any field can contain 0, 1, or more values as long as all the values have the same data type)
 - *Join* (a field that creates a parent-child relationship between documents)
 - *Geo* (a geometric shape with real-world coordinates)
 - *Vector* (a numeric array that contains a point in a multi-dimensional space)

More on Data Types

- Elasticsearch infers the schema from incoming document (dynamic mapping), but setting the `dynamic` parameter to “*strict*” in the mapping makes Elasticsearch reject any document that does not conform to the schema defined in the mapping
- A field can be indexed in multiple ways (a TV series episode name -such as “The Sopranos-S06E20-The Blue Comet”- can be indexed as *keyword* to be able to sort series and seasons by name, and indexed as *text* to be able to perform a full text search it)

Changing Mappings of an Existing Index

- Once an index is created, its mappings can be modified only within limits: fields can be added and some limited mappings parameter can be updated, but nothing more
- If the index mappings have to be changed in other ways (say, removing some fields), the only way is to *reindex* an index, which is a fancy name for the creation of a new index, the insertion of all documents from the old index, and the removal of the old index.
- The `_reindex` endpoint makes the process a bit easier

`POST _reindex`

```
{
  "source": {
    "index": "my_index_v1"
  },
  "dest": {
    "index": "my_index_v2"
  }
}
```

Inserting Documents

- When documents are inserted, some metadata fields are added (by default `_id` and `_index`)

Common *metadata fields* include:

- `_index` (the index the document belongs to)
 - `_id` (document ID - similar to the primary key in a relational DBMS)
 - `_version` (document version)
 - `_primary_term` (it identifies the primary shard)
 - `_seq_no` (the change number within the primary shard)
 - `_source` (the original JSON document)
 - `_routing` (the shard the document is assigned to - only when the routing is forced when creating the document)
- By default the search API does not return all metadata fields, but it adds `_score` (the relevancy of a document in a query: the higher the score, the more relevant the document within the query results)
 - The ID of a document is assigned by Elasticsearch if no ID is given in the request
 - The ID is unique to a document

Retrieving Documents

- The main task of a DBMS is to answer questions about the database, that is to allow data *querying*
- Elasticsearch can be queried using different query languages
 - **Query DSL**: the most widely used query language in Elasticsearch, it is a superset of the Apache Lucene query syntax expressed in JSON
 - **SQL**: a re-framing of *Query DSL* in SQL syntax (it implements a very limited subset of SQL)
 - **EQL**: a specialized language for querying events and finding correlations between them
 - **ES|QL**: (ElasticSearch Piped Query Language) is a language that resembles SQL but where clauses are concatenated via pipes (“|”) and may include transformations
- This course will focus on *SQL* and *Query DSL*, with some notions on ES|QL

The Elasticsearch SQL

- There are many limitations to this dialect of SQL:
 - It is not a full-featured SQL (no real joins, etc) as in a relational DBMS such PostgreSQL or MySQL
 - It has limited geographic queries support (only distance between points is supported)
 - `DISTINCT` is not supported
 - Multiple-values of a field cannot be returned

...and so on
- *Query DSL* syntax is unwieldy (it is JSON) while *SQL* is easier to understand
- Since *Query DSL* is more powerful than *SQL*, there are two techniques that can be used to get around its limitations:
 - Translate an *SQL* query into a *Query DSL* one and then add the parts that cannot be performed using *SQL* (the `/_sql/translate` API endpoint does it)
 - Add the `filter` clause (it contains a *Query DSL* expression) to an *SQL* query

Updating and Deleting Documents

- Updating and deleting documents is done via a ReSTful API with the HTTP methods **PUT** and **DELETE**
- Since Elasticsearch is concurrent, there may be requests that arrive out-of-order (say, when a node fails), therefore the document *sequence number* and the *primary term* should be sent along with the update or delete request.
 - When these two meta fields are omitted, the update or delete request may return status 409 (conflict) or risk overwriting previous changes.
 - The meta fields can be obtained by reading the document just before sending the update or delete request
 - 409s can happen especially during search updates or deletes, as other clients may have changed the documents between the search and update/delete steps (ElasticSearch does not lock data as relational DBMS do: no “**SELECT FOR UPDATE**” in ElasticSearch)

Querying Multiple Indexes at Once

- Indexes can be grouped, as already mentioned, in a data stream and be queried in a single query, provided their names follow the same pattern and have similar schema
- There is an alternative method that does not suffer from these limitations
- In Query DSL, comma-separated indexes can be specified in the URL:
`GET /index1,index2,index3/_search`
- In Elasticsearch SQL, multiple indexes can be specified in the FROM clause:
`SELECT *
FROM "index1", "index2", "index3"
WHERE field = 'value'`
- Querying multiple indexes in a single query is more efficient than sequential querying each index in a separate query

Query DSL: Multiple Criteria Search #1

- Example of a query to select all H1 students that are named “John” (“must” means “and”). By default Query DSL is case-insensitive for text fields

POST /students/_search

```
{
  "query": {
    "bool": {
      "must": [
        {
          "range": {
            "mark": {
              "gte": 80
            }
          }
        },
        {
          "match": {
            "name": "john *"
          }
        }
      ]
    }
  }
}
```

Query DSL: Multiple Criteria Search #2

- Example of a query to select all students that have a score higher than 90 or are named “John” (“should” means “or”)

POST /students/_search

```
{
  "query": {
    "bool": {
      "should": [
        {
          "range": {
            "mark": {
              "gte": 90
            }
          }
        },
        {
          "match": {
            "name": "john *"
          }
        }
      ]
    }
  }
}
```

Query DSL: Geographic Search

- Example of a query to select all observations that are within a rectangle

```
POST /filebeat-8.7.1/_search
```

```
{
  "query": {
    "bool": {
      "filter": {
        "geo_shape": {
          "location": {
            "shape": {
              "type": "envelope",
              "coordinates": [ [ 13.0, -53.0 ], [ 14.0, -52.0 ] ]
            },
            "relation": "within"
          }
        }
      }
    }
  }
}
```

SQL: Geographic Search

```
POST /_sql?format=text
{
  "query": "SELECT data.status, data.type FROM \"filebeat-8.7.1\"",
  "filter": {
    "geo_shape": {
      "location": {
        "shape": {
          "type": "envelope",
          "coordinates": [ [ 13.0, -53.0 ], [ 14.0, -52.0 ] ]
        },
        "relation": "within"
      }
    }
  }
}
```

- The “trick” is to use a Query DSL *filter* in place of the **WHERE** clause, which makes the request easier to read
- Note the use of:
 - Escaped double quotes (`\`) to get around invalid characters in SQL
 - *Dot notation* (`data.type`) to reference nested JSON fields

SQL: Text Search

- Matches messages that contain “Windows”

```
POST /_sql?format=txt {  
  "query":  
    "SELECT data.status, data.type FROM \"filebeat-8.7.1\" WHERE MATCH('message',  
    'Windows')"  
}
```

- Same query in Query DSL (much harder to read!):

```
{  
  "query": {  
    "match": {  
      "message": "Windows"  
    }  
  },  
  "fields": [ {  
    "field": "data.status"  
  }, {  
    "field": "data.type"  
  }  
]  
}
```

SQL: Keyword Search

- Just an ordinary equality (note the use of `LCASE` function because the `data.type` field is of type keyword)

```
POST /_sql?format=txt
```

```
{  
  "query": "SELECT data.status FROM \"filebeat-8.7.1\" WHERE LCASE(data.type) =  
  'chatcompletions-request'  
}
```

SQL: Time Search

- Elasticsearch has flexible time functions
- Select observations that are older than two days ago

```
POST /_sql?format=txt
```

```
{  
  "query": "SELECT \"@timestamp\", data.status FROM \"filebeat-8.7.1\"  
WHERE \"@timestamp\" < TODAY() - INTERVAL '2' DAYS"  
}
```

- Select observations that have been logged in the last four months

```
POST /_sql?format=txt
```

```
{  
  "query": "SELECT DATE_PART('year', \"@timestamp\") AS D, data.status  
FROM \"filebeat-8.7.1\" WHERE \"@timestamp\" > (TODAY() - INTERVAL '4'  
MONTH) GROUP BY D, data.status"  
}
```

SQL: Pagination #1

- Pagination (by default Elasticsearch returns 1,000 elements) is implemented via a *cursor* (think of it as a bookmark that marks a page in a book)

```
POST /_sql?format=json
```

```
{  
  "query": "SELECT HISTOGRAM(\"@timestamp\", INTERVAL 1 HOUR) AS H,  
COUNT(*) AS N FROM \"filebeat-8.7.1\" GROUP BY H"  
}
```

```
{  
  "columns": [...],  
  "rows": [...],  
  "cursor":  
"t8jsA0RGTACKUlu00zAUtduRZqY/bCWhFTP9IyVx0iXOKA87jf+a0LSN8xJ1Jo81sBrEA1gJ4  
pNdQALMaD6QCuJI1q3rc+2jcw9kAMYAwuu4zKvydJTJ5f59WVdRB8B3CK8WN7cv+U2SwN1recy  
Tk9z1FQAQD0t6/koZkZ8ghGyo/MS3SziD4K1vMhA/fPzyFYDJ2DQ80uDFI3nc4Sc4BZ0BN57HW  
wA/X8jb9KQ1WFi521sBqyKTHlmAFMvf13a3aC3/rsEprFv1DiNJdYHtq61KFuhyGxLbFY11cP  
Op3YX9XYRbF3FDWINZai1zdOCF11L3yiSGAcTG8u3eMt7dxtrv7AiDxo0aHoIPbR5CFJK4rWQU  
e8efd30CF1ueK0d0ebUWB+1hY/aeiR4dW8oip1nHsmdLhFOQ1MuHHWZWcPff6+N/Ke20be7575  
1SKhFrLoW9xGjJ12H21L1BZNEVNT6J9+cc9r0czPSVmy98w+2rb0KZBuBi8rAKJwzw37HPHU+D  
CjwA95TQ6qYZoQRfkhSeu/mqud1RLJG08Dsd9KmUDx1EVzxnUzGGD8rwousLPZ/Zk3BDwAAAP/  
/AwA="
```

SQL: Pagination #2

- The cursor ID has to be passed to subsequent request to retrieve the next page

```
POST /_sql?format=json
```

```
{  
  "query": "SELECT HISTOGRAM(\"@timestamp\", INTERVAL 1 HOUR) AS H,  
COUNT(*) AS N FROM \"filebeat-8.7.1\" GROUP BY H",  
  "cursor":  
  "t8jsA0RGTAckUlu00zAUtduRZqY/bCWhFTP9IyVx0iXOKA87jf+a0LSN8xJ1Jo81sBrEA1gJ4  
pNdQALMaD6QCuJI1q3rc+2jcw9kAMYAwuu4zKvydJTJ5f59WVdRB8B3CK8WN7cv+U2SwNlrecy  
Tk9z1FQAQD0t6/koZkZ8ghGyo/MS3SziD4K1vMhA/fPzyFYDJ2DQ80uDFI3nc4Sc4BZ0BN57HW  
wA/X8jb9KQ1WFi521sBqyKTHlmAFMvf13a3aC3/rsEprFv1DiNJdYHtq61KFuhyGxLbFY11cP  
Op3YX9XYRbF3FDWINZai1zd0CF1lL3yiSGAcTG8u3eMt7dxtrv7AiDxo0aHoIPbR5CFJK4rWQU  
e8efd30CF1ueK0d0ebUWB+1hY/aeiR4dW8oip1nHsmdLhFOQ1MuHHWZWcPff6+N/Ke20be7575  
1SKhFrLoW9xGjJl2H21L1BZNEVNT6J9+cc9r0czPSVmy98w+2rb0KZBuBi8rAKJwzw37HPHU+D  
CjwA95TQ6qYZoQRfkhSeu/mqudlRLJG08Dsd9KmUDx1EVzxnUzGGD8rwousLPZ/Zk3BDwAAAP/  
/AwA="  
}
```

- When the cursor field is empty all the results have been returned
- Cursors are time-sensitive and are deleted after a few minutes

SQL: Metric Aggregation

- The usual SQL aggregate functions

```
POST /_sql?format=txt
```

```
{  
  "query": "SELECT DATE_PART('year', \"@timestamp\") AS D, data.status,  
COUNT(*) AS N FROM \"filebeat-8.7.1\" WHERE \"@timestamp\" > (TODAY() -  
INTERVAL '4' YEARS) GROUP BY D, data.status"  
}
```

```
POST /_sql?format=txt
```

```
{  
  "query": "SELECT data.status AS STATUS, SUM(data.payload_size) AS SIZE  
FROM \"filebeat-8.7.1\" GROUP BY data.status"  
}
```

SQL: Bucket Aggregation

- A bucket aggregation is a way to group documents into sets (likes bars of an histogram). Buckets aggregations can have sub-aggregations and are more powerful than metric aggregations (in the following query `1 YEAR` is the interval for each histogram “bar”)

```
POST /_sql?format=txt
```

```
{  
  "query": "SELECT HISTOGRAM(CAST(date AS TIMESTAMP), INTERVAL 1 YEAR) AS D,  
COUNT(*) AS N FROM temperatures GROUP BY D"  
}
```

D	N
2012-01-01T00:00:00.000Z	91
2013-01-01T00:00:00.000Z	364
2014-01-01T00:00:00.000Z	365

SQL: Support for OLAP (sort of)

- Elasticsearch SQL has the capability of producing contingency tables using the **PIVOT** clause. It transforms some values (in this example **rel_hum**) into columns and computes one metric (average **air_temp**) for all different value of another columns (**stationid**) for all the rows having a given value in the pivoted column (**rel_hum**)

POST /_sql?format=txt

```
{  
  "query": "SELECT * FROM (SELECT rel_hum, stationid, air_temp FROM  
observations) PIVOT (AVG(air_temp) FOR rel_hum IN (65.0, 66.0, 67.0))"  
}
```

stationid	65.0	66.0	67.0
95936	19.78888914320204	19.542857306344168	18.30000008477105

- This is a nice feature, but Elasticsearch is no substitute for an OLAP engine

EQL: Looking for Matching Events

- This is an optional feature and not included in the deployment we will use during the workshop
- EQL is a language used to match events (*Event Query Language*) that occur near each other, often used to search logs for suspicious activity
- The following query retrieve events that happen within one minute from each other and satisfy a logical expression

```
GET /my-data-stream/_eql/search
```

```
{  
  "query": ""  
    sequence with maxspan=1m  
      [ process where process.name == "cmd.exe" ]  
      ![ process where stringContains(process.command_line,  
"ocx") ]  
      [ file where stringContains(file.name, "scrobj.dll") ]  
    ""  
}
```

(Example taken from the Elasticsearch documentation)

ES|QL: Piped Data and Transformations

- This query language is the most recent addition to Elasticsearch, and is expected to be the most performing and flexible to query and aggregate data
- ES|QL is very loosely based on the SQL syntax, but relies on the concept of piped data, that is, every pipe character “|” signals the output of the preceding clause is fed into the next clause:

```
FROM logs
| WHERE level == "ERROR" AND duration_ms > 500
| EVAL duration_sec = duration_ms / 1000.0
| STATS
    count(*)           AS error_count,
    avg(duration_sec)  AS avg_duration_sec,
    max(duration_sec)  AS max_duration_sec
BY service
| SORT error_count DESC
| LIMIT 5
```

- NOTE: “==” is the equality comparison operator, while “=” is the assignment operator

Joins in Elasticsearch

- Elasticsearch is not a relational DBMS, hence joins (matching data from different tables in the same query) are supported only in a limited way and only one-to-many (many-to-many joins are not supported)
 - Parent-child relationship declared in the mappings of an index. This establishes a relationship between two document belonging to the same index (supported only in Query DSL).
 - Enriched data: fields from different indexes can be merged in the same query (supported only in ES|QL)
 - Lookup indexes: similar to enriched data, but simpler and more in line with the joins of a relational DBMS (supported only in ES|QL)

Joins: Parent-child

- Establishing the relationship between two documents within the same index can be done in the mappings (one “course” document has many “student” documents as children)

```
"mappings": {  
  "properties": {  
    "uomid": {  
      "type": "text"  
    },  
    "name": {  
      "type": "text"  
    },  
    "mark": {  
      "type": "short"  
    },  
    "coursename": {  
      "type": "text"  
    },  
    "relation_type": {  
      "type": "join",  
      "relations": {  
        "course": "student"  
      }  
    }  
  }  
}
```

Joins: Parent-child When Inserting

- When a document is added, the relationship has to be explicitly set between the two documents. In addition both the parent and the children have to be in the same shard, hence the “routing” parameter must be provided when inserting a document

```
{
  "_id": "comp90024",
  "coursedescription": "Cloud Computing",
  "relation_type": {
    "name": "course"
  }
}

{
  "_id": 841,
  "name": "John Smith",
  "uomid": "1234567",
  "mark": 80,
  "relation_type": {
    "name": "student",
    "parent": "comp90024"
  }
}
```

Joins: Parent-child in Query DSL

- Query to select all the children of parent that has the name field equal to “comp90024” (a document labelled as “course”) with Query DSL

```
GET /search {
  "query": {
    "bool": {
      "must": [
        {
          "range": {
            "mark": {
              "gt": 80
            }
          }
        }, {
          "has_parent": {
            "parent_type": "course",
            "query": {
              "match": {
                "coursename": "Cloud Computing"
              }
            }
          }
        }
      ]
    }
  }
}
```

Lookup Joins

- Lookup indexes are meant to join a large index with smaller *lookup data*, which are restricted to one shard and to a number of documents that fit in the heap memory (as a rule-of-thumb, in the tens of thousand, or tens of MBs)
- To join a main index with a *lookup index* in ES|QL, the `LOOKUP JOIN` clause is used (note that the `KEEP` clause lists data from both indexes)

```
FROM logs  
  | LOOKUP JOIN users ON user_id  
  | KEEP @timestamp, user_id, user_name, department, message
```

- This join makes sense only if the number of documents in the `users` index is limited

Vector Data

- Vectors (both sparse and dense) can be loaded into Elasticsearch and queried for *equality* and *k-Nearest Neighbours* searches
- During the mapping definition, a vector is defined with its type, no. of dimensions, the distance to use and some index parameters

```
{  
  "type": "dense_vector",  
  "dims": 5,  
  "similarity": "l2_norm"  
}
```

Vector Data Query

- Elasticsearch uses the *Hierarchical Navigable Small World* algorithm for vector searches

- Query example taken from the Elasticsearch documentation.

POST `byte-image-index/_search`

```
{  
  "knn": {  
    "field": "byte-image-vector",  
    "query_vector": [-5, 9],  
    "k": 10,  
    "num_candidates": 100  
  },  
  "fields": [ "title" ]  
}
```

- NOTE: `num_candidates` is a search parameters that restricts the search in each shard to the given number of candidate documents

Application of Vector Data

- **Machine Learning:** vectors are especially useful to store *features*
- **Retrieval-Augmented Generation:** A natural language question can be encoded into a *vector embedding*, then a search executed to find the nearest k vectors in a corpus of text. The corresponding texts can then be fed into a Large Language Model to give some context to the user's question

Developing with Elasticsearch

- The main interaction is with the ReSTful API, but there is an SQL CLI tool to query the databases

- Client libraries:

- Elasticsearch Python Client
- Elasticsearch JavaScript Client
- and more...

Active clients

- [Java Client](#)
- [JavaScript Client](#)
- [Ruby Client](#)
- [Go Client](#)
- [.NET Client](#)
- [PHP Client](#)
- [Perl Client](#)
- [Python Client](#)
- [Eland Client](#)
- [Rust Client](#)

- Kibana

The screenshot displays the Elasticsearch Dev Tools interface. On the left, a sidebar contains a 'Management' section with a gear icon and a 'Dev Tools' section. The main area is titled 'Console' and includes tabs for 'Search Profiler', 'Grok Debugger', and 'Painless Lab' (marked as BETA). Below these tabs are sub-tabs for 'History', 'Settings', 'Variables', and 'Help'. The console shows a POST request to the endpoint `temperatures/_search?format=yaml` with a JSON body. The response is a YAML-formatted JSON object. The request body includes a `knn` query for the `temperature` field, a `query_vector` of 18 floats, `k: 10`, `num_candidates: 100`, and `fields: [ "date" ]`. The response includes metadata like `took: 6`, `timed_out: false`, `_shards: { total: 3, successful: 3, skipped: 0, failed: 0 }`, and a `hits` section with `total: { value: 10, relation: "eq" }`, `max_score: 0.9999888`, and a single hit with `_index: "temperatures"` and `_id: "vEU51I0BVliBERVri"`.

```
1 # Click the Variables button, above, to create your own variables.
2 POST temperatures/_search?format=yaml
3 {
4   "knn": {
5     "field": "temperature",
6     "query_vector": [274.62,275.18,275.9,276.74,277.65,278.56,279.4,280.12,280.68,
7                     ,281.03,281.15,281.03,280.68,280.12,279.4,278.56,277.65,276.74,275.9,275.18,
8                     ,274.62,274.27,274.15,274.27],
9     "k": 10,
10    "num_candidates": 100
11  },
12  "fields": [ "date" ]
13 }
14
15 ---
16 took: 6
17 timed_out: false
18 _shards:
19   total: 3
20   successful: 3
21   skipped: 0
22   failed: 0
23 hits:
24   total:
25     value: 10
26     relation: "eq"
27   max_score: 0.9999888
28   hits:
29     - _index: "temperatures"
30       _id: "vEU51I0BVliBERVri"
```

Bibliography

- [1] *3D Data Management: Controlling Data Volume, Velocity, and Variety*, Doug Laney, Gartner Group, 2001
- [2] *Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services*, Seth Gilbert, 2001
- [3] *ElasticSearch in Action - 2nd Edition*, Madhusudhan Konda, 2023. Chapters: 1.4, 2.3, 2.4, 2.5, 2.6, 3.2, 3.5, 3.6, 4.3, 4.5, 4.6, 6.8, 6.9, 12.1, 12.2, 12.3, 12.4, 13.2, 13.3, 14.1, 14.3, 15.3
- [4] *Getting Started with ElasticStack 8.0*, Asjad Athick, 2022. Chapters: 3
- [5] *Vector Search with OpenAI Embeddings: Lucene Is All You Need*, Jimmy Lin, R. Pradeep et al., 2023